

Studentu rezultatu analizes programa
v2.0

Generated by Doxygen 1.14.0

1 Studentų rezultatų analizės programa (v0.4)	1
1.1 Programos struktūra	1
1.2 Sugeneruoti duomenų failai	1
1.3 1 tyrimas – failų generavimo sparta, kai ND kiekis = 3	2
1.4 2 tyrimas – duomenų apdorojimo sparta, kai ND kiekis = 3 ir rušiuojama pagal rezultatą	2
1.5 Rezultatų analizė	2
1.6 Rezultatų nuotraukos	2
1.7 Konteinerių testavimas	2
1.8 Visi konteineriai buvo tikrinami tais pačiais programos sugeneruotais failais ir rušiuoji pagal vidurkj, ir buvo įvertintas jų darbo laikas.	2
1.9 Vector	2
1.10 Deque	3
1.11 List	3
1.12 Rezultatai	3
1.13 1 Strategija	3
1.14 Vector skirstymas	3
1.15 Deque skirstymas	3
1.16 List skirstymas	4
1.17 Rezultatai	4
1.18 2 Strategija	4
1.19 Vector skirstymas	4
1.20 Deque skirstymas	4
1.20.1 List skirstymas	4
1.20.2 3 Strategija	5
1.20.3 Vector skirstymas (buvo pasiimtas 1 strategijos budas ir panaudotas copy_if)	5
1.20.4 Deque skirstymas (buvo pasiimtas 1 strategijos budas ir panaudotas copy_if)	5
1.20.5 List skirstymas (buvo pasiimtas 1 strategijos būdas ir panaudotas copy_if)	5
1.20.6 v.pradine release	5
1.20.7 v0.1	6
1.20.8 v0.2	6
1.20.9 v0.3	6
1.20.10 v0.4	6
1.20.11 v1.0	6
1.20.12 Kompiliavimas ir paleidimas	6
1.20.12.1 Naudojant Makefile	6
1.20.13 Kompiliuoti terminale:	6
1.20.14 Svarbu: norint kompiliuoti deque ar list failą, privalote pakeisti konteinerio tipą į atitinkamą isvestis.h ir isvesti.cpp failuose!!!	6
1.20.15 Class ir Struct palyginimas	6
1.20.16 Struct	6
1.20.17 Class	7
1.20.18 Optimizavimo flagų palyginimas	7
1.20.18.1 Failas: studentai1000000.txt	7

1.20.18.2 Failas: studentai100000.txt	7
1.20.19 v1.2	7
1.20.19.1 Rule of Five realizacija	7
1.20.19.2 Duomenų įvestis	8
1.20.19.3 Duomenų išvestis	8
1.20.19.4 Perkrauti operatoriai	8
1.20.20 v1.5	8
1.20.20.1 Abstrakti klasė Zmogus	8
1.20.21 v2.0	8
2 Hierarchical Index	9
2.1 Class Hierarchy	9
3 Class Index	11
3.1 Class List	11
4 File Index	13
4.1 File List	13
5 Class Documentation	15
5.1 Asmuo Struct Reference	15
5.1.1 Member Data Documentation	15
5.1.1.1 pavarde	15
5.1.1.2 vardas	15
5.2 Studentas Class Reference	15
5.2.1 Detailed Description	17
5.2.2 Constructor & Destructor Documentation	17
5.2.2.1 Studentas() [1/4]	17
5.2.2.2 Studentas() [2/4]	17
5.2.2.3 Studentas() [3/4]	17
5.2.2.4 Studentas() [4/4]	17
5.2.2.5 ~Studentas()	18
5.2.3 Member Function Documentation	18
5.2.3.1 addPaz()	18
5.2.3.2 clearPaz()	18
5.2.3.3 exam()	18
5.2.3.4 med()	18
5.2.3.5 operator=() [1/2]	18
5.2.3.6 operator=() [2/2]	18
5.2.3.7 pavarde()	18
5.2.3.8 paz()	18
5.2.3.9 print()	19
5.2.3.10 rez()	19
5.2.3.11 setExam()	19

5.2.3.12 setMed()	19
5.2.3.13 setPavarde()	19
5.2.3.14 setRez()	19
5.2.3.15 setVardas()	19
5.2.3.16 vardas()	19
5.2.4 Friends And Related Symbol Documentation	20
5.2.4.1 operator<<	20
5.2.4.2 operator>>	20
5.3 Zmogus Class Reference	20
5.3.1 Detailed Description	21
5.3.2 Constructor & Destructor Documentation	21
5.3.2.1 Zmogus() [1/2]	21
5.3.2.2 Zmogus() [2/2]	21
5.3.2.3 ~Zmogus()	21
5.3.3 Member Function Documentation	21
5.3.3.1 pavarde()	21
5.3.3.2 print()	21
5.3.3.3 vardas()	21
5.3.4 Member Data Documentation	21
5.3.4.1 pavarde_	21
5.3.4.2 vardas_	21
6 File Documentation	23
6.1 generavimas.cpp File Reference	23
6.1.1 Function Documentation	23
6.1.1.1 generuotiFaila()	23
6.2 generavimas.h File Reference	23
6.2.1 Function Documentation	24
6.2.1.1 generuotiFaila()	24
6.3 generavimas.h	24
6.4 isvesti.cpp File Reference	24
6.4.1 Function Documentation	24
6.4.1.1 spausdinti()	24
6.5 isvestis.h File Reference	25
6.5.1 Function Documentation	25
6.5.1.1 spausdinti()	25
6.6 isvestis.h	25
6.7 ivestis.cpp File Reference	25
6.7.1 Function Documentation	26
6.7.1.1 arTikRaides()	26
6.7.1.2 ivestiSveika()	26
6.8 ivestis.h File Reference	26

6.8.1 Function Documentation	26
6.8.1.1 arTikRaides()	26
6.8.1.2 iverstiSveika()	26
6.9 iverstis.h	26
6.10 main_deque.cpp File Reference	27
6.10.1 Function Documentation	27
6.10.1.1 automatiskai()	27
6.10.1.2 generavimas()	27
6.10.1.3 main()	27
6.10.1.4 ranka()	27
6.10.1.5 skaitymas()	28
6.11 main_list.cpp File Reference	28
6.11.1 Function Documentation	28
6.11.1.1 automatiskai()	28
6.11.1.2 generavimas()	28
6.11.1.3 main()	29
6.11.1.4 ranka()	29
6.11.1.5 skaitymas()	29
6.12 main_vector.cpp File Reference	29
6.12.1 Function Documentation	29
6.12.1.1 automatiskai()	29
6.12.1.2 generavimas()	30
6.12.1.3 main()	30
6.12.1.4 ranka()	30
6.12.1.5 skaitymas()	30
6.13 README.md File Reference	30
6.14 skaiciavimai.cpp File Reference	30
6.14.1 Function Documentation	30
6.14.1.1 skaiciuotiGalutini()	30
6.14.1.2 skaiciuotiGalutiniMed()	30
6.14.1.3 skaiciuotiMediana()	31
6.15 skaiciavimai.h File Reference	31
6.15.1 Function Documentation	31
6.15.1.1 skaiciuotiGalutini()	31
6.15.1.2 skaiciuotiGalutiniMed()	31
6.15.1.3 skaiciuotiMediana()	31
6.16 skaiciavimai.h	31
6.17 studentas.cpp File Reference	32
6.17.1 Function Documentation	32
6.17.1.1 operator<<()	32
6.17.1.2 operator>>()	32
6.18 studentas.h File Reference	32

6.19 studentas.h	33
6.20 test_student.cpp File Reference	33
6.20.1 Function Documentation	34
6.20.1.1 main()	34
6.21 test_zmogus.cpp File Reference	34
6.21.1 Function Documentation	34
6.21.1.1 main()	34
6.22 zmogus.h File Reference	34
6.23 zmogus.h	35
Index	37

Chapter 1

Studentų rezultatų analizės programa (v0.4)

Programa skirta studentų duomenų apdorojimui. Ji gali:

- generuoti studentų failus su atsitiktiniais pažymiais
- nuskaityti studentų duomenis iš failo
- apskaičiuoti galutinį balą
- surūšiuoti studentus pagal pasirinktą kriterijų
- padalinti studentus į dvi grupes:
 - vargšiukai (galutinis < 5.0)
 - galvočiai (galutinis ≥ 5.0)
- išvesti rezultatus į naujus failus

1.1 Programos struktūra

Projektas suskirstytas į kelis failus:

- `main.cpp` – pagrindinė programos logika
- `studentas.h` – `Studentas` struktūra
- `skaiciavimai.cpp` / `.h` – galutinio balo ir medianos skaičiavimas
- `investis.cpp` / `.h` – įvesties validacija
- `isvestis.cpp` / `.h` – rezultatų spausdinimas
- `generavimas.cpp` / `.h` – failų generavimas

1.2 Sugeneruoti duomenų failai

Programos testavimui buvo sugeneruoti šie failai:

	Failas	Studentų skaičius
	studentai1000.txt	1 000
	studentai10000.txt	10 000
	studentai100000.txt	100 000
	studentai1000000.txt	1 000 000
	studentai10000000.txt	10 000 000

1.3 1 tyrimas – failų generavimo sparta, kai ND kiekis = 3

Buvo matuojamas laikas, reikalingas sugeneruoti studentų failus.

1.4 2 tyrimas – duomenų apdorojimo sparta, kai ND kiekis = 3 ir rušiuojama pagal rezultatą

Buvo matuojamas:

- failo nuskaitymo laikas
- rūšiavimo laikas
- studentų skirstymo laikas
- rezultatų įrašymo laikas
- bendras programos veikimo laikas

1.5 Rezultatų analizė

Didėjant studentų skaičiui programos veikimo laikas proporcingai didėja. Didžiausią dalį vykdymo laiko užima duomenų nuskaitymas ir rūšiavimas.

Failų generavimas taip pat tampa žymiai lėtesnis su labai dideliais duomenų kiekiais.

1.6 Rezultatų nuotraukos

<https://prnt.sc/XKvfu3bsZgqw>

<https://prnt.sc/00aFDSNmRW8w>

1.7 Konteinerių testavimas

1.8 Visi konteineriai buvo tikrinami tais pačiais programos sugeneruotais failais ir rušiuoji pagal vidurkį, ir buvo įvertintas jų darbo laikas.

1.9 Vector

	Irašų skaičius	Nuskaitymas (s)	Rūšiavimas (s)	Skirstymas (s)
	1 000	0.0031935	0.0011885	0.00057
	10 000	0.0676135	0.0127499	0.0048795
	100 000	0.262755	0.184618	0.0530411
	1 000 000	2.83759	2.39984	0.56462
	10 000 000	25.4285	22.1732	4.14701

<https://prnt.sc/xUvOjmgTLny>

1.10 Deque

	Irašų skaičius	Nuskaitymas (s)	Rūšiavimas (s)	Skirstymas (s)
	1 000	0.0027926	0.0012407	0.0002988
	10 000	0.0233315	0.0163861	0.0032776
	100 000	0.256054	0.269708	0.0424128
	1 000 000	2.67485	3.92968	0.604689
	10 000 000	26.1153	33.8854	3.99703

<https://prnt.sc/l9anm9YWxfQT>

1.11 List

	Irašų skaičius	Nuskaitymas (s)	Rūšiavimas (s)	Skirstymas (s)
	1 000	0.0044035	0.0005226	0.0005889
	10 000	0.0411307	0.0075837	0.0073089
	100 000	0.374854	0.100233	0.092111
	1 000 000	3.84726	1.82138	0.926026
	10 000 000	41.8413	19.7203	8.03509

<https://prnt.sc/YI2qr-p5Lctg>

1.12 Rezultatai

Atlikus testavimus su skirtingais konteineriais (`std::vector`, `std::deque`, `std::list`) pastebėta, kad bendras programos našumas labiausiai priklauso nuo naudojamo konteinerio tipo. `std::vector` pasižymėjo geriausiu bendru veikimo laiku, ypač rūšiavimo operacijoje, `std::deque` rezultatai buvo labai panašūs į `vector`, tačiau šiek tiek lėtesni rūšiavime. `std::list` kai kuriais atvejais parodė geresnius rezultatus rūšiavimo metu, tačiau bendras veikimo laikas buvo didesnis.

1.13 1 Strategija

1.14 Vector skirstymas

| Irašų skaičius | Skirstymas (s) | | 1 000 | 0.00057 | | 10 000 | 0.0048795 | | 100 000 | 0.0530411 | | 1 000 000 | 0.56462 | | 10 000 000 | 4.14701 |

1.15 Deque skirstymas

	Įrašų skaičius	Skirstymas (s)
	1 000	0.0002988
	10 000	0.0032776
	100 000	0.0424128
	1 000 000	0.604689
	10 000 000	3.99703

1.16 List skirstymas

	Įrašų skaičius	Skirstymas (s)
	1 000	0.0005889
	10 000	0.0073089
	100 000	0.092111
	1 000 000	0.926026
	10 000 000	8.03509

1.17 Rezultatai

Vector ir deque skirsto panašiai, o list užtrunka gerokai ilgiau.

1.18 2 Strategija

1.19 Vector skirstymas

	Įrašų skaičius	Skirstymas (s)
	1 000	0.0124654
	10 000	1.28622
	100 000	132.325

| 1 000 000 | ?? | – Užtruko per ilgai | 10 000 000 | ?? | – Užtruko per ilgai

https://prnt.sc/IICk56_DyjhH

1.20 Deque skirstymas

	Įrašų skaičius	Skirstymas (s)
	1 000	0.0208023
	10 000	1.94427
	100 000	210.984

| 1 000 000 | ?? | – Užtruko per ilgai | 10 000 000 | ?? | – Užtruko per ilgai

<https://prnt.sc/TQfSkokNC5Fm>

1.20.1 List skirstymas

	Irašų skaičius	Skirstymas (s)
	1 000	0.0003278
	10 000	0.0031034
	100 000	0.0465449
	1 000 000	0.530851
	10 000 000	6.45129

<https://prnt.sc/HyYIXIkFzfGb>

1.20.2 3 Strategija

1.20.3 Vector skirstymas (buvo pasiimtas 1 strategijos budas ir panaudotas copy_if)

	Irašų skaičius	Skirstymas (s)
	1 000	0.0002801
	10 000	0.0029163
	100 000	0.0312289
	1 000 000	0.377198
	10 000 000	4.07662

<https://prnt.sc/6bhs9RWqRGfk>

1.20.4 Deque skirstymas (buvo pasiimtas 1 strategijos budas ir panaudotas copy_if)

	Irašų skaičius	Skirstymas (s)
	1 000	0.0002588
	10 000	0.0021332
	100 000	0.0257383
	1 000 000	0.312913
	10 000 000	3.57977

https://prnt.sc/LxD3_dhFPzLT

1.20.5 List skirstymas (buvo pasiimtas 1 strategijos būdas ir panaudotas copy_if)

	Irašų skaičius	Skirstymas (s)
	1 000	0.00067
	10 000	0.0063254
	100 000	0.11172
	1 000 000	0.904371
	10 000 000	10.5409

https://prnt.sc/sVjqs_UKpsWK

1.20.6 v.pradine release

Sukurtas ir padarytas initial realese.

1.20.7 v0.1

Studentų duomenų apdorojimo programa su vidurkio/medianos skaičiavimu, realizuota naudojant `std::vector`, su atsitiktinių duomenų generavimu.

1.20.8 v0.2

Pridėtas duomenų nuskaitymas iš failo, studentų rūšiavimas pagal pasirinktus kriterijus ir testavimas su dideliais duomenų failais.

1.20.9 v0.3

Atliktas programos refaktorizavimas: kodas išskaidytas į kelis `.cpp` ir `.h` failus, panaudotos struktūros bei pridėtas minimalus išimčių valdymas duomenų ir failų tikrinimui.

1.20.10 v0.4

Pridėtas studentų failų generatorius, realizuotas studentų skirstymas į dvi grupes, rezultatų išvedimas į atskirus failus ir atlikta programos spartos analizė su skirtingo dydžio duomenų failais.

1.20.11 v1.0

Atliktas `std::vector`, `std::list` ir `std::deque` konteinerių našumo tyrimas, palygintos studentų skirstymo strategijos.

1.20.12 Kompiliavimas ir paleidimas

1.20.12.1 Naudojant Makefile

Projektas turi paruoštą `Makefile`.

Norint sukompiliuoti programos versijas:

```
```bash make vector make deque make list
```

### 1.20.13 Kompiliuoti terminale:

```
g++ main_vector.cpp skaiciavimai.cpp ivestis.cpp isvesti.cpp generavimas.cpp -o vector g++ main_deque.cpp
skaiciavimai.cpp ivestis.cpp isvesti.cpp generavimas.cpp -o deque g++ main_list.cpp skaiciavimai.cpp ivestis.cpp
isvesti.cpp generavimas.cpp -o list
```

```
./vector.exe ./deque.exe ./list.exe
```

### 1.20.14 Svarbu: norint kompiliuoti deque ar list failą, privalote pakeisti konteinerio tipą į atitinkamą `isvestis.h` ir `isvesti.cpp` failuose!!!

### 1.20.15 Class ir Struct palyginimas

### 1.20.16 Struct

	Įrašų skaičius	Nuskaitymas (s)	Rūšiavimas (s)	Skirstymas (s)
	1 000 000	2.83759	2.39984	0.56462
	10 000 000	25.4285	22.1732	4.14701

<https://prnt.sc/xUvOjmogTLny>

### 1.20.17 Class

	Įrašų skaičius	Nuskaitymas (s)	Rūšiavimas (s)	Skirstymas (s)
	1 000 000	4.43761	6.83953	1.01807
	10 000 000	54.1757	91.1667	9.09333

<https://prnt.sc/2Wn2nqr0MBxP>

### 1.20.18 Optimizavimo flagų palyginimas

#### 1.20.18.1 Failas: studentai1000000.txt

	Flag	Bendras laikas (s)	EXE dydis (KB)
	O1	8.6348	246.79
	O2	7.60203	239.42
	O3	7.90414	260.7

[https://prnt.sc/\\_SvENR5k\\_TZ8](https://prnt.sc/_SvENR5k_TZ8)

#### 1.20.18.2 Failas: studentai100000.txt

	Flag	Bendras laikas (s)	EXE dydis (KB)
	O1	1.70314	246.79
	O2	2.12781	239.42
	O3	1.5851	260.7

<https://prnt.sc/adRonQMFNraW>

Pastaba: optimizavimo flagų poveikis priklauso nuo konkretaus duomenų kiekio ir programos realizacijos, todėl skirtingiems failų dydžiams greičiausias variantas gali skirtis.

### 1.20.19 v1.2

Šioje versijoje `Studentas` klasei realizuota penkių metodų taisyklė (Rule of Five), įvesties/išvesties operatoriai ir sukurtas rankinis testavimo failas.

#### 1.20.19.1 Rule of Five realizacija

		Metodas	
		Destruktorius <code>~Studentas()</code>	Objektas sunaikinamas
	Kopijavimo konstruktorius <code>Studentas(const Studentas&amp; other)</code>		Sukuria naują objektą kopijuojant iš kito
	Kopijavimo priskyrimo operatorius <code>operator=(const Studentas&amp; other)</code>		Priskiria vieno objekto reikšmės kitam objektui
	Perkėlimo konstruktorius <code>Studentas(Studentas&amp;&amp; other) noexcept</code>		Sukuria objektą perkeldami iš kito
	Perkėlimo priskyrimo operatorius <code>operator=(Studentas&amp;&amp; other) noexcept</code>		Perkelia vieno objekto duomenis kitam objektui

### 1.20.19.2 Duomenų įvestis

Būdas	Aprašymas
Rankiniu būdu	Naudotojas gali įvesti duomenis per klaviatūrą naudojant <code>std::cin</code> ir operatorius <code>&gt;&gt;</code>
Automatinis generavimas	Duomenys generuojami naudojant <a href="#">generavimas.cpp</a>
Iš failo	Duomenys nuskaityti iš <code>.txt</code> failų

### 1.20.19.3 Duomenų išvestis

Būdas	Aprašymas
I ekraną	Duomenys išvedami naudojant <code>std::cout</code> ir operatorius <code>&lt;&lt;</code>
I failą	Rezultatai įrašomi į failus naudojant <code>isvestis.cpp</code>

### 1.20.19.4 Perkrauti operatoriai

`Studentas` klasėje realizuoti operatoriai:

Operatorius	Paskirtis
<code>operator&gt;&gt;</code>	Leidžia įvesti studento duomenis iš įvesties srauto
<code>operator&lt;&lt;</code>	Leidžia išvesti studento duomenis į išvesties srautą

<https://prnt.sc/bo7Rt40WibG4>

## 1.20.20 v1.5

Šioje versijoje programa išplėsta panaudojant paveldėjimą ir abstrakčias klases.

### 1.20.20.1 Abstrakti klasė `Zmogus`

Sukurta bazinė klasė `Zmogus`, skirta bendrai aprašyti žmogų. Ši klasė yra abstrakti.

<https://prnt.sc/4RWDRDKI190Z>

## 1.20.21 v2.0

Šioje versijoje pridėta Doxygen dokumentacija, HTML ir LaTeX katalogai, sukompiliuotas `refman.pdf` failas bei realizuoti unit testai su `assert`.



## Chapter 2

# Hierarchical Index

### 2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Asmuo . . . . .	15
Zmogus . . . . .	20
Studentas . . . . .	15



## Chapter 3

# Class Index

### 3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

<a href="#">Asmuo</a>	.....	15
<a href="#">Studentas</a>	Studento duomenų saugojimo ir apdorojimo klasė .....	15
<a href="#">Zmogus</a>	Abstrakti bazinė žmogaus klasė .....	20



# Chapter 4

## File Index

### 4.1 File List

Here is a list of all files with brief descriptions:

<a href="#">generavimas.cpp</a>	23
<a href="#">generavimas.h</a>	23
<a href="#">isvesti.cpp</a>	24
<a href="#">isvestis.h</a>	25
<a href="#">investis.cpp</a>	25
<a href="#">investis.h</a>	26
<a href="#">main_deque.cpp</a>	27
<a href="#">main_list.cpp</a>	28
<a href="#">main_vector.cpp</a>	29
<a href="#">skaiciavimai.cpp</a>	30
<a href="#">skaiciavimai.h</a>	31
<a href="#">studentas.cpp</a>	32
<a href="#">studentas.h</a>	32
<a href="#">test_student.cpp</a>	33
<a href="#">test_zmogus.cpp</a>	34
<a href="#">zmogus.h</a>	34



## Chapter 5

# Class Documentation

### 5.1 Asmuo Struct Reference

```
#include <studentas.h>
```

#### Public Attributes

- std::string [vardas](#)
- std::string [pavarde](#)

#### 5.1.1 Member Data Documentation

##### 5.1.1.1 pavarde

```
std::string Asmuo::pavarde
```

##### 5.1.1.2 vardas

```
std::string Asmuo::vardas
```

The documentation for this struct was generated from the following file:

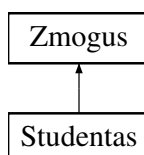
- [studentas.h](#)

### 5.2 Studentas Class Reference

Studento duomenų saugojimo ir apdorojimo klasė.

```
#include <studentas.h>
```

Inheritance diagram for Studentas:



## Public Member Functions

- [Studentas](#) ()
- [Studentas](#) (const std::string &[vardas](#), const std::string &[pavarde](#), const std::vector< int > &[paz](#), int [exam](#), double [rez](#)=0.0, double [med](#)=0.0)
- [Studentas](#) (const [Studentas](#) &other)
 

*Kopijavimo konstruktorius.*
- [Studentas](#) ([Studentas](#) &&other) noexcept
 

*Perkélimo konstruktorius.*
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other) noexcept
- [~Studentas](#) ()
 

*Studento destruktorius.*
- void [print](#) () const override
 

*Išveda studento informaciją.*
- std::string [vardas](#) () const
- std::string [pavarde](#) () const
- const std::vector< int > & [paz](#) () const
- int [exam](#) () const
- double [rez](#) () const
- double [med](#) () const
- void [setVardas](#) (const std::string &[vardas](#))
- void [setPavarde](#) (const std::string &[pavarde](#))
- void [setExam](#) (int [exam](#))
- void [setRez](#) (double [rez](#))
- void [setMed](#) (double [med](#))
- void [addPaz](#) (int [paz](#))
- void [clearPaz](#) ()

## Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()=default
- [Zmogus](#) (const std::string &[vardas](#), const std::string &[pavarde](#))
- virtual [~Zmogus](#) ()=default
- std::string [vardas](#) () const
- std::string [pavarde](#) () const

## Friends

- std::istream & [operator>>](#) (std::istream &in, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &s)

## Additional Inherited Members

## Protected Attributes inherited from [Zmogus](#)

- std::string [vardas\\_](#)
- std::string [pavarde\\_](#)



### 5.2.1 Detailed Description

Studento duomenų saugojimo ir apdorojimo klasė.

Klasė paveldi abstrakčią klasę [Zmogus](#). Realizuota Rule of Five.

### 5.2.2 Constructor & Destructor Documentation

#### 5.2.2.1 Studentas() [1/4]

```
Studentas::Studentas ()
```

#### 5.2.2.2 Studentas() [2/4]

```
Studentas::Studentas (
 const std::string & vardas,
 const std::string & pavarde,
 const std::vector< int > & paz,
 int exam,
 double rez = 0.0,
 double med = 0.0)
```

#### 5.2.2.3 Studentas() [3/4]

```
Studentas::Studentas (
 const Studentas & other)
```

Kopijavimo konstruktorius.

##### Parameters

<i>other</i>	Kitas <a href="#">Studentas</a> objektas.
--------------	-------------------------------------------

#### 5.2.2.4 Studentas() [4/4]

```
Studentas::Studentas (
 Studentas && other) [noexcept]
```

Perkėlimo konstruktorius.

##### Parameters

<i>other</i>	Perkeliamas objektas.
--------------	-----------------------

#### 5.2.2.5 ~Studentas()

```
Studentas::~~Studentas ()
```

Studento destruktorius.

### 5.2.3 Member Function Documentation

#### 5.2.3.1 addPaz()

```
void Studentas::addPaz (
 int paz)
```

#### 5.2.3.2 clearPaz()

```
void Studentas::clearPaz ()
```

#### 5.2.3.3 exam()

```
int Studentas::exam () const
```

#### 5.2.3.4 med()

```
double Studentas::med () const
```

#### 5.2.3.5 operator=() [1/2]

```
Studentas & Studentas::operator= (
 const Studentas & other)
```

#### 5.2.3.6 operator=() [2/2]

```
Studentas & Studentas::operator= (
 Studentas && other) [noexcept]
```

#### 5.2.3.7 pavarde()

```
std::string Studentas::pavarde () const
```

#### 5.2.3.8 paz()

```
const std::vector< int > & Studentas::paz () const
```

### 5.2.3.9 print()

```
void Studentas::print () const [override], [virtual]
```

Išveda studento informaciją.

Implements [Zmogus](#).

### 5.2.3.10 rez()

```
double Studentas::rez () const
```

### 5.2.3.11 setExam()

```
void Studentas::setExam (
 int exam)
```

### 5.2.3.12 setMed()

```
void Studentas::setMed (
 double med)
```

### 5.2.3.13 setPavarde()

```
void Studentas::setPavarde (
 const std::string & pavarde)
```

### 5.2.3.14 setRez()

```
void Studentas::setRez (
 double rez)
```

### 5.2.3.15 setVardas()

```
void Studentas::setVardas (
 const std::string & vardas)
```

### 5.2.3.16 vardas()

```
std::string Studentas::vardas () const
```

## 5.2.4 Friends And Related Symbol Documentation

### 5.2.4.1 operator<<

```
std::ostream & operator<< (
 std::ostream & out,
 const Studentas & s) [friend]
```

### 5.2.4.2 operator>>

```
std::istream & operator>> (
 std::istream & in,
 Studentas & s) [friend]
```

The documentation for this class was generated from the following files:

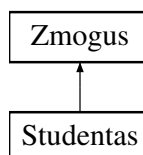
- [studentas.h](#)
- [studentas.cpp](#)

## 5.3 Zmogus Class Reference

Abstrakti bazinė žmogaus klasė.

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:



### Public Member Functions

- [Zmogus](#) ()=default
- [Zmogus](#) (const std::string &[vardas](#), const std::string &[pavarde](#))
- virtual [~Zmogus](#) ()=default
- std::string [vardas](#) () const
- std::string [pavarde](#) () const
- virtual void [print](#) () const =0

*Gryna virtuali spausdinimo funkcija.*

### Protected Attributes

- std::string [vardas\\_](#)
- std::string [pavarde\\_](#)

### 5.3.1 Detailed Description

Abstrakti bazinė žmogaus klasė.

### 5.3.2 Constructor & Destructor Documentation

#### 5.3.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [default]
```

#### 5.3.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
 const std::string & vardas,
 const std::string & pavarde) [inline]
```

#### 5.3.2.3 ~Zmogus()

```
virtual Zmogus::~Zmogus () [virtual], [default]
```

### 5.3.3 Member Function Documentation

#### 5.3.3.1 pavarde()

```
std::string Zmogus::pavarde () const [inline]
```

#### 5.3.3.2 print()

```
virtual void Zmogus::print () const [pure virtual]
```

Gryna virtuali spausdinimo funkcija.

Implemented in [Studentas](#).

#### 5.3.3.3 vardas()

```
std::string Zmogus::vardas () const [inline]
```

### 5.3.4 Member Data Documentation

#### 5.3.4.1 pavarde\_

```
std::string Zmogus::pavarde_ [protected]
```

#### 5.3.4.2 vardas\_

```
std::string Zmogus::vardas_ [protected]
```

The documentation for this class was generated from the following file:

- [zmogus.h](#)



## Chapter 6

# File Documentation

### 6.1 generavimas.cpp File Reference

```
#include <fstream>
#include <iomanip>
#include <random>
#include <stdexcept>
#include <string>
#include <chrono>
#include <iostream>
#include "generavimas.h"
```

#### Functions

- void [generuotiFaila](#) (const std::string &failoPavadinimas, int studentuKiekis, int ndKiekis)

#### 6.1.1 Function Documentation

##### 6.1.1.1 generuotiFaila()

```
void generuotiFaila (
 const std::string & failoPavadinimas,
 int studentuKiekis,
 int ndKiekis)
```

### 6.2 generavimas.h File Reference

```
#include <string>
```

## Functions

- void [generuotiFaila](#) (const std::string &failoPavadinimas, int studentuKiekis, int ndKiekis)

### 6.2.1 Function Documentation

#### 6.2.1.1 generuotiFaila()

```
void generuotiFaila (
 const std::string & failoPavadinimas,
 int studentuKiekis,
 int ndKiekis)
```

## 6.3 generavimas.h

[Go to the documentation of this file.](#)

```
00001 #ifndef GENERAVIMAS_H
00002 #define GENERAVIMAS_H
00003
00004 #include <string>
00005
00006 void generuotiFaila(const std::string& failoPavadinimas, int studentuKiekis, int ndKiekis);
00007
00008 #endif
```

## 6.4 isvesti.cpp File Reference

```
#include <iostream>
#include <iomanip>
#include <deque>
#include <list>
#include "isvestis.h"
```

## Functions

- void [spausdinti](#) (const std::vector< [Studentas](#) > &grupe, bool rodytiMediana)

### 6.4.1 Function Documentation

#### 6.4.1.1 spausdinti()

```
void spausdinti (
 const std::vector< Studentas > & grupe,
 bool rodytiMediana)
```



## 6.5 isvestis.h File Reference

```
#include <vector>
#include <deque>
#include <list>
#include "studentas.h"
```

### Functions

- void `spausdinti` (const std::vector< `Studentas` > &grupe, bool rodytiMediana)

### 6.5.1 Function Documentation

#### 6.5.1.1 spausdinti()

```
void spausdinti (
 const std::vector< Studentas > &grupe,
 bool rodytiMediana)
```

## 6.6 isvestis.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include <vector>
00003 #include <deque>
00004 #include <list>
00005 #include "studentas.h"
00006
00007 void spausdinti(const std::vector<Studentas>& grupe, bool rodytiMediana);
```

## 6.7 ivestis.cpp File Reference

```
#include "ivestis.h"
#include <cctype>
#include <iostream>
```

### Functions

- bool `arTikRaides` (const std::string &tekstas)
- int `ivestiSveika` (const std::string &pranesimas)

## 6.7.1 Function Documentation

### 6.7.1.1 arTikRaides()

```
bool arTikRaides (
 const std::string & tekstas)
```

### 6.7.1.2 ivestiSveika()

```
int ivestiSveika (
 const std::string & pranesimas)
```

## 6.8 ivestis.h File Reference

```
#include <string>
```

### Functions

- bool [arTikRaides](#) (const std::string &tekstas)
- int [investiSveika](#) (const std::string &pranesimas)

## 6.8.1 Function Documentation

### 6.8.1.1 arTikRaides()

```
bool arTikRaides (
 const std::string & tekstas)
```

### 6.8.1.2 ivestiSveika()

```
int ivestiSveika (
 const std::string & pranesimas)
```

## 6.9 ivestis.h

[Go to the documentation of this file.](#)

```
00001 #ifndef IVESTIS_H
00002 #define IVESTIS_H
00003
00004 #include <string>
00005
00006 bool arTikRaides(const std::string& tekstas);
00007 int investiSveika(const std::string& pranesimas);
00008
00009 #endif // IVESTIS_H
```

## 6.10 main\_deque.cpp File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <utility>
#include <random>
#include <fstream>
#include <sstream>
#include <cctype>
#include <stdexcept>
#include <chrono>
#include <deque>
#include "studentas.h"
#include "skaiciavimai.h"
#include "investis.h"
#include "isvestis.h"
#include "generavimas.h"
```

### Functions

- int [ranka](#) ()
- int [generavimas](#) ()
- int [automatiskai](#) ()
- int [skaitymas](#) ()
- int [main](#) ()

### 6.10.1 Function Documentation

#### 6.10.1.1 automatiskai()

```
int automatiskai ()
```

#### 6.10.1.2 generavimas()

```
int generavimas ()
```

#### 6.10.1.3 main()

```
int main ()
```

#### 6.10.1.4 ranka()

```
int ranka ()
```

#### 6.10.1.5 skaitymas()

```
int skaitymas ()
```

### 6.11 main\_list.cpp File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <utility>
#include <random>
#include <fstream>
#include <sstream>
#include <cctype>
#include <stdexcept>
#include <list>
#include <chrono>
#include "studentas.h"
#include "skaiciavimai.h"
#include "investis.h"
#include "isvestis.h"
#include "generavimas.h"
```

#### Functions

- int [ranka](#) ()
- int [generavimas](#) ()
- int [automatiskai](#) ()
- int [skaitymas](#) ()
- int [main](#) ()

#### 6.11.1 Function Documentation

##### 6.11.1.1 automatiskai()

```
int automatiskai ()
```

##### 6.11.1.2 generavimas()

```
int generavimas ()
```

### 6.11.1.3 main()

```
int main ()
```

### 6.11.1.4 ranka()

```
int ranka ()
```

### 6.11.1.5 skaitymas()

```
int skaitymas ()
```

## 6.12 main\_vector.cpp File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <utility>
#include <random>
#include <fstream>
#include <sstream>
#include <cctype>
#include <stdexcept>
#include <chrono>
#include "studentas.h"
#include "skaiciavimai.h"
#include "investis.h"
#include "isvestis.h"
#include "generavimas.h"
```

### Functions

- int [ranka](#) ()
- int [generavimas](#) ()
- int [automatiskai](#) ()
- int [skaitymas](#) ()
- int [main](#) ()

### 6.12.1 Function Documentation

#### 6.12.1.1 automatiskai()

```
int automatiskai ()
```

#### 6.12.1.2 generavimas()

```
int generavimas ()
```

#### 6.12.1.3 main()

```
int main ()
```

#### 6.12.1.4 ranka()

```
int ranka ()
```

#### 6.12.1.5 skaitymas()

```
int skaitymas ()
```

## 6.13 README.md File Reference

## 6.14 skaiciavimai.cpp File Reference

```
#include "skaiciavimai.h"
#include <algorithm>
```

### Functions

- double [skaiciuotiMediana](#) (std::vector< int > paz)
- double [skaiciuotiGalutini](#) (int suma, int kiekis, int egzaminas)
- double [skaiciuotiGalutiniMed](#) (std::vector< int > paz, int egzaminas)

### 6.14.1 Function Documentation

#### 6.14.1.1 skaiciuotiGalutini()

```
double skaiciuotiGalutini (
 int suma,
 int kiekis,
 int egzaminas)
```

#### 6.14.1.2 skaiciuotiGalutiniMed()

```
double skaiciuotiGalutiniMed (
 std::vector< int > paz,
 int egzaminas)
```

### 6.14.1.3 skaiciuotiMediana()

```
double skaiciuotiMediana (
 std::vector< int > paz)
```

## 6.15 skaiciavimai.h File Reference

```
#include <vector>
```

### Functions

- double [skaiciuotiMediana](#) (std::vector< int > paz)
- double [skaiciuotiGalutini](#) (int suma, int kiekis, int egzaminas)
- double [skaiciuotiGalutiniMed](#) (std::vector< int > paz, int egzaminas)

### 6.15.1 Function Documentation

#### 6.15.1.1 skaiciuotiGalutini()

```
double skaiciuotiGalutini (
 int suma,
 int kiekis,
 int egzaminas)
```

#### 6.15.1.2 skaiciuotiGalutiniMed()

```
double skaiciuotiGalutiniMed (
 std::vector< int > paz,
 int egzaminas)
```

#### 6.15.1.3 skaiciuotiMediana()

```
double skaiciuotiMediana (
 std::vector< int > paz)
```

## 6.16 skaiciavimai.h

[Go to the documentation of this file.](#)

```
00001 #ifndef SKAICIAVIMAI_H
00002 #define SKAICIAVIMAI_H
00003
00004 #include <vector>
00005
00006 double skaiciuotiMediana(std::vector<int> paz);
00007 double skaiciuotiGalutini(int suma, int kiekis, int egzaminas);
00008 double skaiciuotiGalutiniMed(std::vector<int> paz, int egzaminas);
00009
00010 #endif // SKAICIAVIMAI_H
```

## 6.17 studentas.cpp File Reference

```
#include "studentas.h"
#include <utility>
#include <iostream>
```

### Functions

- `std::ostream & operator<<` (`std::ostream &out`, `const Studentas &s`)
- `std::istream & operator>>` (`std::istream &in`, `Studentas &s`)

### 6.17.1 Function Documentation

#### 6.17.1.1 `operator<<()`

```
std::ostream & operator<< (
 std::ostream & out,
 const Studentas & s)
```

#### 6.17.1.2 `operator>>()`

```
std::istream & operator>> (
 std::istream & in,
 Studentas & s)
```

## 6.18 studentas.h File Reference

```
#include <string>
#include <vector>
#include <iostream>
#include "zmogus.h"
```

### Classes

- class `Studentas`  
*Studento duomenų saugojimo ir apdorojimo klasė.*
- struct `Asmuo`



## 6.19 studentas.h

[Go to the documentation of this file.](#)

```

00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <iostream>
00007 #include "zmogus.h"
00008
00016
00017 class Studentas : public Zmogus {
00018 private:
00019 std::vector<int> paz_;
00020 int exam_;
00021 double rez_;
00022 double med_;
00023
00024 public:
00025 Studentas();
00026 Studentas(const std::string& vardas, const std::string& pavarde,
00027 const std::vector<int>& paz, int exam,
00028 double rez = 0.0, double med = 0.0);
00029
00034 Studentas(const Studentas& other);
00039 Studentas(Studentas&& other) noexcept;
00040 Studentas& operator=(const Studentas& other);
00041 Studentas& operator=(Studentas&& other) noexcept;
00045 ~Studentas();
00046
00050 void print() const override;
00051
00052 // getteriai
00053 std::string vardas() const;
00054 std::string pavarde() const;
00055 const std::vector<int>& paz() const;
00056 int exam() const;
00057 double rez() const;
00058 double med() const;
00059
00060 // setteriai
00061 void setVardas(const std::string& vardas);
00062 void setPavarde(const std::string& pavarde);
00063 void setExam(int exam);
00064 void setRez(double rez);
00065 void setMed(double med);
00066 void addPaz(int paz);
00067 void clearPaz();
00068
00069 friend std::istream& operator>(std::istream& in, Studentas& s);
00070 friend std::ostream& operator<(std::ostream& out, const Studentas& s);
00071 };
00072
00073 struct Asmuo {
00074 std::string vardas;
00075 std::string pavarde;
00076 };
00077
00078 #endif

```

## 6.20 test\_student.cpp File Reference

```

#include "studentas.h"
#include <iostream>
#include <vector>
#include <cassert>
#include <sstream>

```

### Functions

- int [main](#) ()

## 6.20.1 Function Documentation

### 6.20.1.1 main()

```
int main ()
```

## 6.21 test\_zmogus.cpp File Reference

```
#include "studentas.h"
#include <iostream>
#include <type_traits>
#include <cassert>
```

### Functions

- int [main](#) ()

## 6.21.1 Function Documentation

### 6.21.1.1 main()

```
int main ()
```

## 6.22 zmogus.h File Reference

```
#include <string>
```

### Classes

- class [Zmogus](#)  
*Abstrakti bazinė žmogaus klasė.*

## 6.23 zmogus.h

[Go to the documentation of this file.](#)

```
00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00010
00011 class Zmogus {
00012 protected:
00013 std::string vardas_;
00014 std::string pavarde_;
00015
00016 public:
00017 Zmogus() = default;
00018 Zmogus(const std::string& vardas, const std::string& pavarde)
00019 : vardas_(vardas), pavarde_(pavarde) {}
00020
00021 virtual ~Zmogus() = default;
00022
00023 // getteriai
00024 std::string vardas() const { return vardas_; }
00025 std::string pavarde() const { return pavarde_; }
00026
00030 virtual void print() const = 0;
00031 };
00032
00033 #endif // ZMOGUS_H
```



# Index

- ~Studentas
  - Studentas, [17](#)
- ~Zmogus
  - Zmogus, [21](#)
- addPaz
  - Studentas, [18](#)
- arTikRaides
  - investis.cpp, [26](#)
  - investis.h, [26](#)
- Asmuo, [15](#)
  - pavarde, [15](#)
  - vardas, [15](#)
- automatiskai
  - main\_deque.cpp, [27](#)
  - main\_list.cpp, [28](#)
  - main\_vector.cpp, [29](#)
- clearPaz
  - Studentas, [18](#)
- exam
  - Studentas, [18](#)
- generavimas
  - main\_deque.cpp, [27](#)
  - main\_list.cpp, [28](#)
  - main\_vector.cpp, [29](#)
- generavimas.cpp, [23](#)
  - generuotiFaila, [23](#)
- generavimas.h, [23](#)
  - generuotiFaila, [24](#)
- generuotiFaila
  - generavimas.cpp, [23](#)
  - generavimas.h, [24](#)
- isvesti.cpp, [24](#)
  - spausdinti, [24](#)
- isvestis.h, [25](#)
  - spausdinti, [25](#)
- investis.cpp, [25](#)
  - arTikRaides, [26](#)
  - investiSveika, [26](#)
- investis.h, [26](#)
  - arTikRaides, [26](#)
  - investiSveika, [26](#)
- investiSveika
  - investis.cpp, [26](#)
  - investis.h, [26](#)
- main
  - main\_deque.cpp, [27](#)
  - main\_list.cpp, [28](#)
  - main\_vector.cpp, [29](#)
- main\_deque.cpp, [27](#)
  - automatiskai, [27](#)
  - generavimas, [27](#)
  - main, [27](#)
  - ranka, [27](#)
  - skaitymas, [27](#)
- main\_list.cpp, [28](#)
  - automatiskai, [28](#)
  - generavimas, [28](#)
  - main, [28](#)
  - ranka, [29](#)
  - skaitymas, [29](#)
- main\_vector.cpp, [29](#)
  - automatiskai, [29](#)
  - generavimas, [29](#)
  - main, [30](#)
  - ranka, [30](#)
  - skaitymas, [30](#)
- med
  - Studentas, [18](#)
- operator<<
  - Studentas, [20](#)
  - studentas.cpp, [32](#)
- operator>>
  - Studentas, [20](#)
  - studentas.cpp, [32](#)
- operator=
  - Studentas, [18](#)
- pavarde
  - Asmuo, [15](#)
  - Studentas, [18](#)
  - Zmogus, [21](#)
- pavarde\_
  - Zmogus, [21](#)
- paz
  - Studentas, [18](#)
- print
  - Studentas, [18](#)
  - Zmogus, [21](#)
- ranka
  - main\_deque.cpp, [27](#)
  - main\_list.cpp, [29](#)

- main\_vector.cpp, 30
- README.md, 30
- rez
  - Studentas, 19
- setExam
  - Studentas, 19
- setMed
  - Studentas, 19
- setPavarde
  - Studentas, 19
- setRez
  - Studentas, 19
- setVardas
  - Studentas, 19
- skaiciavimai.cpp, 30
  - skaiciuotiGalutini, 30
  - skaiciuotiGalutiniMed, 30
  - skaiciuotiMediana, 30
- skaiciavimai.h, 31
  - skaiciuotiGalutini, 31
  - skaiciuotiGalutiniMed, 31
  - skaiciuotiMediana, 31
- skaiciuotiGalutini
  - skaiciavimai.cpp, 30
  - skaiciavimai.h, 31
- skaiciuotiGalutiniMed
  - skaiciavimai.cpp, 30
  - skaiciavimai.h, 31
- skaiciuotiMediana
  - skaiciavimai.cpp, 30
  - skaiciavimai.h, 31
- skaitymas
  - main\_deque.cpp, 27
  - main\_list.cpp, 29
  - main\_vector.cpp, 30
- spausdinti
  - isvesti.cpp, 24
  - isvestis.h, 25
- Studentas, 15
  - ~Studentas, 17
  - addPaz, 18
  - clearPaz, 18
  - exam, 18
  - med, 18
  - operator<<, 20
  - operator>>, 20
  - operator=, 18
  - pavarde, 18
  - paz, 18
  - print, 18
  - rez, 19
  - setExam, 19
  - setMed, 19
  - setPavarde, 19
  - setRez, 19
  - setVardas, 19
  - Studentas, 17
  - vardas, 19
- studentas.cpp, 32
  - operator<<, 32
  - operator>>, 32
- studentas.h, 32
- Studentų rezultatų analizės programa (v0.4), 1
- test\_student.cpp, 33
  - main, 34
- test\_zmogus.cpp, 34
  - main, 34
- vardas
  - Asmuo, 15
  - Studentas, 19
  - Zmogus, 21
- vardas\_
  - Zmogus, 21
- Zmogus, 20
  - ~Zmogus, 21
  - pavarde, 21
  - pavarde\_, 21
  - print, 21
  - vardas, 21
  - vardas\_, 21
  - Zmogus, 21
- zmogus.h, 34